NESNE YÖNELİMLİ PROGRAMLAMA

Ders 9: Struct, Enumeration, Exception Handling, Attributes, Reflection

Konular

- Struct
- Exception Handling
- Enumeration
- Attributes
- Reflection

Struct (Yapı)

- C#, bir yapı değer tipli (value type) bir veri tipidir.
- Özellikle ilişkili farklı veri tiplerini tek değişkende tutmakta yardımcı olur. (Sınıfa benziyor.)
- Struct anahtar kelimesi ile bir yapı kurulur.
- Yapılar kayıtların içeriklerini belirlemek için kullanılır. Örneğin bir kütüphanedeki kitaplar (books) için bir uygulama düşünelim. Bu uygulamada aşağıdaki içerikleri ön plana çıkar:
  - Title (Başlık)
  - Author (Yazar)
  - Subject (Konu)
  - Book ID (Kitap ID)

```
struct Books
{
    public string title;
    public string author;
    public string subject;
    public int book_id;
};
```

Struct

```
struct Books
{
    public string title;
    public string author;
    public string subject;
    public int book_id;
};
```

```
public class testStructure
{
    public static void Main(string[] args)
    {

        Books Book1;    /* Book1 tanımlandı */
        Books Book2;    /* Book2 tanımlandı */

        /* Book1 özellikleri */
        Book1.title = "C Programming";
        Book1.author = "Nuha Ali";
        Book1.subject = "C Programming Tutorial";
        Book1.book_id = 6495407;

        /* Book2 özellikleri */
        Book2.title = "Telecom Billing";
        Book2.author = "Zara Ali";
        Book2.subject = "Telecom Billing Tutorial";
        Book2.book_id = 6495700;

        /* Book1 içeriği */
        Console.WriteLine("Book 1 title : {0}", Book1.title);
        Console.WriteLine("Book 1 author : {0}", Book1.author);
        Console.WriteLine("Book 1 subject : {0}", Book1.subject);
        Console.WriteLine("Book 1 book_id :{0}", Book1.book_id);

        /* Book2 içeriği */
        Console.WriteLine("Book 2 title : {0}", Book2.title);
        Console.WriteLine("Book 2 author : {0}", Book2.author);
        Console.WriteLine("Book 2 subject : {0}", Book2.subject);
        Console.WriteLine("Book 2 book_id : {0}", Book2.book_id);

        Console.ReadKey();
    }
}
```

C# Struct Özellikleri

- C#'ta Yapılar C ve C++ gibi dillere göre farklılıklar gösterir. C# Yapılarının özellikleri şöyledir:
- Yapılar metotlar (methods), alanlar (fileds), özellikler (properties), operator metotları (operator methods) ve olaylar (events) içerebilir.
- Yapılara constructor tanımlanabilir, ama destructor (yıkıcı) tanımlanamaz. Ancak, yapılara geçerli bir constructor tanımlanmaz. İllaki parametreli bir constructor olmalı.
- Bir struct yaratmada new operatörü kullanıldığı zaman, yaratılmış constructor çağrılmış olur. Sınıflardan farklı olarak, new operatörü kullanmadan da struct'ları kullanmak mümkündür.
- Eğer new operatörü kullanılmazsa, alanlar tanımlanmamış kalır ve nesne tüm alanlar tanımlanmadan kullanılamaz.

C# Struct Özellikleri

- Structures can have defined constructors, but not destructors. However, you cannot define a default constructor for a structure. The default constructor is automatically defined and cannot be changed.
- Sınıflardan farklı olarak yapılar diğer sınıf veya yapılardan kalıtım uygulanamaz. Diğer bir deyişle, yapılar diğer yapılara veya sınıflara base olarak kullanılamaz.
- Bir yapı bir veya birden fazla interface'e uygulayabilir. (DateTime'ı hatırla)
- Yapı üyeleri abstract, virtual veya protected olarak belirlenemez.

Struct Örneği

```
public struct ogrenci_s
{
    public string AdSoyad;
    private int _Not;

    public ogrenci_s(string a, int n)
    {
        AdSoyad = a;
        _Not = n;
        //Not değişkeni kullanılamıyor ama class kullanılabiliyor...
    }

    //structta parametresiniz constructor olmaz.
    //public ogrenci_s(){
    //    AdSoyad = "Test";
    //    _Not = 0;
    //}

    public int Not {
        get { return _Not; }
        set {
            _Not = value;
            if (_Not < 0)
                _Not = 0;
            if (_Not > 100)
                _Not = 100;
        }
    }

    public void Bilgi()
    {
        Console.WriteLine("Adı Soyadı: (struct)" + AdSoyad + " : " + Not);
    }
}//struct sonu
```

Sadece public ve private erişim belirteçlerini destekler.

Constructor

Property

Property

Class vs. Struct

```
public class ogrenci_c
{
    public string AdSoyad;
    private int _Not;
    //Sadece class için tanımlanabilir
    public ogrenci_c(){
        AdSoyad = "Guest";
        Not = 0;
    }

    //struct ile aynı
    public ogrenci_c(string a, int n){
        AdSoyad = a;
        Not = n;//struct'tan farklı olabiliyor
    }

    //struct ile aynı
    public int Not
    {
        get { return _Not; }

        set {
            _Not = value;
            if (_Not < 0)
                _Not = 0;
            if (_Not > 100)
                _Not = 100;
        }
    }
    //struct ile aynı
    public void Bilgi()
    {
        Console.WriteLine("Adı Soyadı: (Class) " + AdSoyad + " : " + Not);
    }
}
```

- Struct'tan farklı olarak Class'ta boş constructor tanımlanabilir.
- Erişim belirteçlerinin tümü kullanılabilir.

Struct ve Class Yaratma

- Main bölümünden önceki struct ve class'ları çağıralım.

```
class Program
{
    static void Main(string[] args)...
}
```

```
ogrenci_s _s1 = new ogrenci_s(); //boş constructor yok ama çalışıyor...
_s1.AdSoyad = "Deneme Öğrencisi";
_s1.Not = 45;
_s1.Bilgi();
ogrenci_s _s2 = new ogrenci_s("Deneme Öğrencisi 2", 55);
_s2.Bilgi();
ogrenci_s _s3;
_s3.AdSoyad = "1111";
Console.WriteLine(_s3.AdSoyad);
//_s3.Not = 34;
//_s3.Bilgi(); //??? struct ile çalışmaz new ihtiyaç var
//strcutları dizi olarak tanımlama
ogrenci_s[] _sdizi = new ogrenci_s[5];
_sdizi[0].AdSoyad = "Deneme 1";
_sdizi[0].Not = 450;
_sdizi[1].AdSoyad = "Deneme 2";
_sdizi[1].Not = -5;
_sdizi[2].AdSoyad = "Deneme 3";
_sdizi[2].Not = 55;

for (int i = 0; i < 5; i++)
    _sdizi[i].Bilgi();
```

Struct İşlemleri

```
ogrenci_c _c1 = new ogrenci_c(); //eğer constructor tanımlamazı yoksa çalışmaz
_c1.AdSoyad = "Deneme Öğrencisi";
_c1.Not = 45;
_c1.Bilgi();

ogrenci_c _c2 = new ogrenci_c("Deneme Öğrencisi 2", 65);
_c2.Bilgi();

/*ogrenci_c _c3;
_c3.AdSoyad = "1111";
Console.WriteLine(_c3.AdSoyad);*/
//Kesinlikle çalışmaz... Nesne yaratılmadığı için (New kullanılması lazım)
//class'ımız statik olsaydı new işlemine gerek olmazdı.
//geçmiş örneklere bakın...
```

Class İşlemleri

Nested Structs (İç içe yapılar)

- Sınıf ve yapılar iç içe kullanılabilir.

```
//iç içe yapılar (nested structure)
public struct S1
{
    public string A;

    public struct S2
    {
        public string B;
    }
}

//iç içe sınıflar (nested class)
public class C1
{
    public string A;

    public class C2
    {
        public string B;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        /*S1 _s1 = new S1();
        _s1.A = "Değer 1";

        S1.S2 _s12 = new S1.S2();
        _s12.B = "İç Değer";*/

        //new kullanmadan
        S1 _sn1;
        _sn1.A = "Değer 1";

        S1.S2 _sn12;
        _sn12.B = "İç Değer";

        //sınıf yaratılmak zorundadır.
        C1 _c1 = new C1();
        C1.C2 _c2 = new C1.C2();
    }
}
```

Hem class hem de struct'ta B'ye sırasıyla C2 ve S2 üzerinden ulaşılabilir.

Struct vs. Class

- Class'la referans tipindendir (reference types)
- Struct'lar ise de değer tipindendir. (value types)
- Yapılar kalıtım / miras desteklemez.
- Yapıların geçerli constructor'a sahip değildir. Diğer deyişle constructor'ları parametre içermek zorundadır.

Struct vs. Class

```
public class sinif
{
    public int x;
}

public struct yapi
{
    public int x;
}
```

- Bu örnekte referans tipinden olan sınıf ile değer tipinden olan yapı yaratılacak ve aktarımlar üzerinden fark anlatılacaktır.

Struct vs. Class

```
public class sinif
{
    public int x;
}

public struct yapi
{
    public int x;
}
```

```
static void Main(string[] args)
{
    sinif _c1 = new sinif();
    _c1.x = 5;

    sinif _c2 = new sinif();
    _c2 = _c1;//bellek üzerine işlem yapılır. Class referans türüdür.

    Console.WriteLine("Class ve Nesne üzerine olan işlemler");
    _c1.x = 8;
    Console.WriteLine(_c2.x.ToString());

    _c2.x = 7;
    _c1.x = 10;
    Console.WriteLine(_c1.x.ToString());
    Console.WriteLine(_c2.x.ToString());

    Console.WriteLine("Struct üzerine olan işlemler");
    yapi _s1, _s2;
    _s1.x = 5;
    _s2 = _s1;
    Console.WriteLine(_s2.x.ToString());

    //Struct değer türüdür. bellek adresleri üzerine değil, direkt bellekteki içerik üzerine işlem yapılır.
    _s1.x = 13;
    _s2.x = 1;
    Console.WriteLine(_s1.x.ToString());
    Console.WriteLine(_s2.x.ToString());

    Console.ReadLine();
}
```

Her iki çıktı sonucu: 10
_c1.x'teki değişim her iki sınıfı etkiler.

Çıktılar sırasıyla: 13 ve 1'dir.
_s1.x'teki değişim sadece kendi yapısını etkiler.

- _c2=_c1 işleminde adres ataması yapılır. Bu aşamadan sonra _c1 üzerindeki her değişiklik _c2'ye yansır. Başka _c1 ve _c2 adresleri eşitlenmiştir.
- _s2 = _s1 işleminde ise sadece değer ataması yapılmıştır. Her iki değişkenin çalışma adresi halen birbirinden farklıdır.

Exception Handling (İstisnai Durum işleme)

- Bir exception (istisnai durum) bir programın çalışması esnasında ortaya çıkan bir problemdir.
  - Sıfıra bölme (divide by zero)
  - Taşma Hatası (Overflow)
  - Yetersiz Hafıza (Out of Memory)
  - Donanımın bulunamaması
  - Kullanıcının yanlış değer girmesi vb.
- İstisnai durumlar, bir program parçasından diğerine geçiş kontrolü için imkan sağlar.
- C#'de istisnai durum işleme için try, catch, finally ve throw anahtar kelimeleri kullanılır. (Benzer durum Java içinde geçerlidir.)

Exception Handling (İstisnai Durum işleme)

- try: bir try bloğu istisnai durumların oluşabileceği kod bloklarını belirler. Bir veya birden fazla catch bloğu ile devam eder.
- catch: İstisnai durum bulunduğunda yapılacak işlemleri içeren kısımdır. Catch bir istisnai durumun yakalandığını gösterir. Eğer Catch kısmındaki kodlar çalışmıyorsa istisnai durum oluşmamıştır. Bir try birden fazla catch içerebilir.
- finally: Bu blok istisnai durumun oluşup oluşmadığına bakmaksızın verilen bir kod kümesini çalıştırmak için kullanılır. Kullanılması seçimlik bir durumdur, yani finally olmayan bir try-catch tanımlanabilir.
- throw: Bir programda bir problem oluştuğu zaman ilgili hata nesnesi sınıfını atmaktır (throw). Bunun için throw anahtar sözcüğü kullanılır.

Exception Handling (İstisnai Durum işleme)

```
try
{
    //Exception oluşturabilecek satırlar
}
catch (ExceptionName e1)
{
    //Hata yakalandığında yapılacak işlem
}
catch (ExceptionName e2)
{
    //Hata yakalandığında yapılacak işlem
}
catch (ExceptionName eN)
{
    //Hata yakalandığında yapılacak işlem
}
finally
{
    //Her halikarda çalıştırılacak satır
}
```

- Farklı hata durumları için birden fazla catch olabilir.
- Tüm hatalı durumları tek catch içinde de toplamanız mümkündür.
- Throw hata oluştuğunda oluşturulacak kısım olduğu için bu kod bloğu içinde yer almamıştır. (İleriki slaytlarda kullanımını göreceğiz.)

Exception Handling

```
class Program
{
    static void Main(string[] args)
    {
        int x = 5;
        int y = 0;
        //sıfıra bölme hatası
        try
        {
            int z = x / y;
            Console.WriteLine(z);
        }
        catch (Exception ex)
        {
            Console.WriteLine("Sıfıra bölme hatası"); //Kişisel çıktı
            Console.WriteLine(ex.Message); //ex değeri içinden dönen çıktı
            Console.WriteLine(ex.Source); //ex değerinin kaynağı
        }

        //karakter içeren bir değeri sayıyı çevirme hatası
        string sayi = "rfdhjfd";
        try
        {
            Convert.ToInt32(sayi);
        }
        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }
}
```

Hatanın oluştuğu kısım, bu aşamadan sonra bağlı catch bloğu devreye girer. Hatanın oluştuğu kısımdaki bir alt satırdaki kod çalışmaz.

Hatanın oluştuğu kısım

Finally bloğu kullanma seçimliktir.

Exception Properties

- Exception özellikleri kullanarak hata hakkında bilgiler edinebiliriz.
  - HelpLink: Genelde boştur, string değer döndürür. Özellikle kendi exception'larınızda hata durumda bir web sayfasına kullanıcığı göndermek için Link tutmada kullanılabilir.
  - Message: Hatanın nedenini kısa bir mesajla verir. Mesaj sadece okunabilir durumdadır. (sadece get var demek ki!)
  - Source: Uygulama ismidir. String değerdir.
  - StackTrace: Derlenen programının dizinidir.
  - TargetSite: Hatanın oluştuğu yerdeki metotun ismidir. Hata kayıtlarını tutmadan yardımcı olur.

C# - Exception Handling

- C# istisnai durumları sınıflarda bulunur.
- C#'ta istisnai durum sınıfları, System.Exception sınıfından direct / indirect olarak oluşur.
- Örneğin System.ApplicationException ve System.SystemException sınıfları System.Exception sınıfından kalıtılmıştır. (derived)
- System.ApplicationException uygulama programları tarafından oluşturulan istisnai durumları destekler. System.SystemException sınıfı ise sistem istinai durumları için tanımlanmış bir base sınıftır.
- The System.ApplicationException class supports exceptions generated by application programs. Hence the exceptions defined by the programmers should derive from this class.

C# - Exception Handling

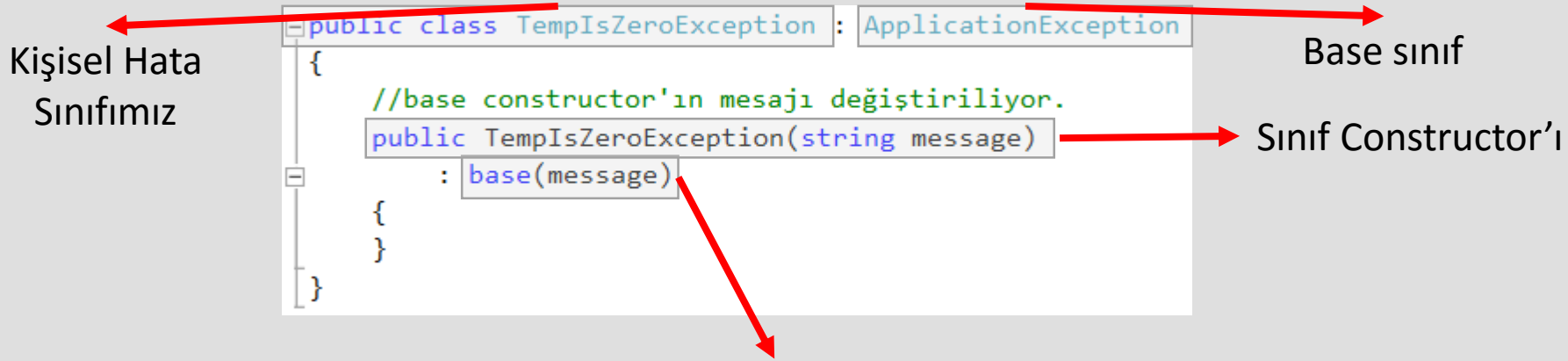
- System.SystemException'dan türemiş bazı istisnai durum sınıfları:

Exception Sınıfı	Tanımı
System.IO.IOException	I / O hatalarını işler. (Dosya yazma / okuma)
System.IndexOutOfRangeException	Bir dizi dışına çıkıldığında oluşan hataları işler.
System.ArrayTypeMismatchException	Bir veri tipi, dizi tipi ile uyuşmadığında oluşan hataları işler.
System.NullReferenceException	Bir null nesneden kaynaklandığında oluşan hataları işler.
System.DivideByZeroException	Sıfıra bölme hatalarını işler.
System.InvalidCastException	Veri tipinden yanlış işlenmesinden kaynaklı hataları işler.
System.OutOfMemoryException	Yetersiz hafıza durumunda üretilen hataları işler.
System.StackOverflowException	Stack taştığında oluşabilecek hataları işler.

Daha fazla hata sınıfı vardır. Uygulamaya göre de farklı hata sınıfları karşınıza çıkabilir.

User-Defined Exceptions (Kullanıcı-Tanımlı)

- Kendi istisnai durumunuzu ayrıca tanımlayabilirsiniz.
- Kullanıcı-tanımlı istinai durum sınıfları ApplicationException sınıftan türetilir.



- Base sınıfın constructor'ı üzerinden string değer gönderme ile base'in verdiği hatayı değiştirme.
- Normal şartlarda ApplicationException sınıfı «Application Error» veya «Uygulama Hatası» gibi hata mesajı verir. Bu örnekte bu hata mesajını değiştiriyoruz.

User-Defined Exceptions (Kullanıcı-Tanımlı)

```
public class TempIsZeroException : ApplicationException
{
    //base constructor'ın mesajı değiştiriliyor.
    public TempIsZeroException(string message)
        : base(message)
    {
    }
}

public class Temperature
{
    public int temperature = 0;

    public void showTemp()
    {
        if (temperature == 0)
        {
            throw (new TempIsZeroException("Zero Temperature found"));
        }
        else
        {
            Console.WriteLine("Temperature: {0}", temperature);
        }
    }
}
```

Throw hata oluşturmak için kullanılır. Throw ile bir nesne tetiklenebilir.

User-Defined Exceptions (Kullanıcı-Tanımlı)

- Eğer direkt showtemp metotu çağrılırsa program hataya düşer. temperature değeri o'dan farklı yapılıp çağrılırsa catch bölümüne düşmeden program çalışır.
- Bu örnekte try-catch kullanımı ile kendi yarattığımız bir exception'ı kullandık.

```
class Program
{
    static void Main(string[] args)
    {
        Temperature temp = new Temperature();
        try
        {
            //temp.temperature = 10;
            temp.showTemp();
        }
        catch (TempIsZeroException e)
        {
            Console.WriteLine("TempIsZeroException: {0}", e.Message);
        }
        Console.ReadKey();
    }
}
```

Kişisel Exception Sınıfımız

Bu string bilgiyi güncellemiştik.

Enumeration (Numaralandırma)

- Bir enumeration tamsayı sabitlerini adlandıran bir kümedir.
- enum anahtar sözcüğü ile bildirilir.
- C# enumeration'lar değer veri tipindedir. Başka bir deyişle, kendine ait bir değeri içerir ve kalıtılamaz.

```
enum <enum_name>
{
    enumeration list
};
```

- «enum_name» enumariton tip adını berlirler.
- «enumeration list» virgülle ayrılmış verileri içerir.

Enumeration (Numaralandırma)

```
using System;

namespace Ders09_06_Enumeration
{
    class Program
    {
        enum Gunler { Pzt = 2, Sl = 4, Car, Per, Cum, Cmt, Pz };

        static void Main(string[] args)
        {
            int x = (int)Gunler.Car;
            Console.WriteLine(x);
            Console.ReadKey();
        }
    }
}
```

Attributes (Nitelikler)

- Nitelikler, uygulandıkları classes, methods, structures, enumerators veya assemblies gibi programınızın parçalarının çalışma zamanındaki davranışlarının değiştirilmesine olanak sağlayan sınıflardır.
- Attribute, abstract bir sınıftır. Dolayısıyla örneklenemez ancak bir nitelik sınıfının içermesi gereken temel üyeleri bünyesinde barındırır.
- .Net içinde mevcut olan veya bizim tarafımızdan geliştirilen nitelikler daima Attribute sınıfından türemek zorundadırlar.
- Üretilen assembly içerisinde yer alan tip ve üyelere ekstra bilgiler katabilmeleridir. Bir başka deyişle, metadata içerisine ilave bilgiler eklenebilmesini sağlamaktadır.
- Bu metadata bilgileri üretilen assembly' lar içerisinde yer alır ve yansıma(Reflection) teknikleri ile çalışma zamanında değerlendirilebilir.
- Bu konu ile ileri de çok karşılaşacaksınız. Hızlı kod üretmede kullanabilirsiniz.

Attributes (Nitelikler)

- .Net Framework'te Niteliklerin iki tip sağlar:
  - pre-defined attributes (Önceden tanımlanmış nitelikler)
    - AttributeUsage
    - Conditional
    - Obsolete
  - custom built attributes (Özel yapılmış nitelikler)
- Nitelikler «[…]» karakterleri arasında çağrılır. Kullanımı:

  [attribute(positional_parameters, name_parameter = value, ...)]

  element

- element: bir class, method, structure, enumerator veya assembly olabilir. Positional_parameters gerekli bilgileri belirtir ve name_parameter istediği bilgileri belirtir.

AttributeUsage

- AttributeUsage kişisel attribute class nasıl olacağını tanımlar. Item'ların type'larına hangi attribute'un uygulanabileceğini belirler.

```
[AttributeUsage(
   validon,
   AllowMultiple=allowmultiple,
   Inherited=inherited
)]
```

- validon parametresi dil elementlerinin hangi attritube yerleştirilebileceğini belirler. Bir enumerator AttributeTargets değerinin bir kombinasyonudur. Geçerli değeri AttributeTargets.All'dur.
- allowmultiple parametresi (Seçimlik - optional) bir Boolean değerdir, bu attritube'un AllowMultiple property değer sağlar. Eğer true ise , attritube çok amaçlıdır. Geçerli değeri false'tur (tek amaçlı).
- inherited paramtresi (Seçimlik) bir Boolean değerdir, bu attritube'un Inherited property değer sağlar. Eğer true ise, derived sınıflar tarafından kalıtım uygulanabilir. Geçerli değeri false'tur (Kalıtım uygulanamaz.).

AttributeUsage

- Kullanımı

```
[AttributeUsage(
   validon,
   AllowMultiple=allowmultiple,
   Inherited=inherited
)]
```

- Örneğin:

```
[AttributeUsage(AttributeTargets.Class |
AttributeTargets.Constructor |
AttributeTargets.Feild |
AttributeTargets.Method |
AttributeTargets.Property,
AllowMultiple = true)]
```

AttributeUsage

```
// Bir attritube sınıflara eklenebilir
[AttributeUsage(AttributeTargets.Class)]
public class PerlsAttribute : Attribute
{
}

//Üstteki sınıfın kısa sözdizimi ile çağrımı
[Perls]
class Example1
{
}

//Üstteki sınıfın uzun sözdizimi ile çağrımı
[PerlsAttribute]
class Example2
{
}

class Program
{
    static void Main()
    {
        //Derleme yapılabilmesi için
    }
}
```

Attritube sınıfı olduğu belirleniyor.

Attritube sınıfından kalıtım uygulanıyor

PerlsAttribute sınıfın attribute özellikleri Example1 ve Example2 sınıflarına aktarılmıştır.

PerlsAttribute yerine Attribute kelimesi kullanılmadan sadece Perls yazılabilir.

AttributeUsage - Constructor

```
// Attribute
[AttributeUsage(AttributeTargets.Class)]
public class PerlsAttribute : Attribute
{
    //Veri depolamak için bir değişken
    string _id;
    //Attritube'in Constructor'ı
    public PerlsAttribute(string id)
    {
        this._id = id;
    }
    //Id getir
    public string Id
    {
        get { return this._id; }
    }
}

//Attritube'i uygula
[Perls("Dot")]
class Example2
{
}
```

constructor

Id değişkenine constructor üzerinden Attribute değeri gönderme

Attribute - Property

```
// Attribute
[AttributeUsage(AttributeTargets.Class)]
public class Perls3Attribute : Attribute
{
    //Veri depolamak için bir değişken
    string _id;
    //Attritube'in Constructor'ı boş
    public Perls3Attribute()
    {
    }
    //Id getir ve değiştirebilir (get / set)
    public string Id
    {
        get { return this._id; }
        set { this._id = value; }
    }
}

//Attritube'i uygula
[Perls3Attribute(Id = "Sam")]
class Example3
{
}
```

Constructor boş

Id bunun üzerinden ulaşırız.

Id değişkenine property üzerinden Attribute değeri gönderme

Conditional

- Conditional (Şartlı) method her zaman çalışmaz.
- #define ile tanımlanan belirli bir ifade sadece derlenir.
- #undef ile istenilmeyen kısımlar çalışmaz.
- #define ve undef işlemi kodun üst kısmına yapılır.
- Conditional söz dizimi:

```
[Conditional(
    conditionalSymbol
)]
```

Conditional

```
#define PERLS
#undef DOT
using System;
using System.Diagnostics;

namespace Ders09_07_Conditional
{
    class Program
    {
        static void Main()
        {
            MethodA();
            MethodB();
            Console.ReadKey();
        }

        [Conditional("PERLS")]
        static void MethodA()
        {
            Console.WriteLine("MethodA çalıştı.");
        }

        [Conditional("DOT")]
        static void MethodB()
        {
            Console.WriteLine("MethodB çalıştı.");
        }
    }
}
```

Tanımlı kısım. Derlenir.

Tanımlanmayan kısım. Derlenmez.

Çalışır

Çalışmaz

MetdodB'yi çalıştırmamak için normal şartlarda main'den silmelisiniz. Fakat yazılım karmaşıklaştıkça bu silme süreci uzayabilir. Bazen derlemek istemediğiniz kısım halen geliştirilmekte olabilir. Bu durumlarda Conditional Metot çok işe yarar.

Obsolete (eski)

- Bu predefined attritube kullanılmayacak olan program varlığını işaretler.
- Belirli bir elementi compiler tarafından derlenmeyeceğine dair bilgi vermeye olanak tanır.
- Örneğin, yeni bir metot yazdığınızda hala eski metodunuzu görmek istiyorsanız (silmek istemiyorsanız), fakat takım arkadaşlarınızı (veya ileride kendinizi) uyarmak için kullanılabilir.

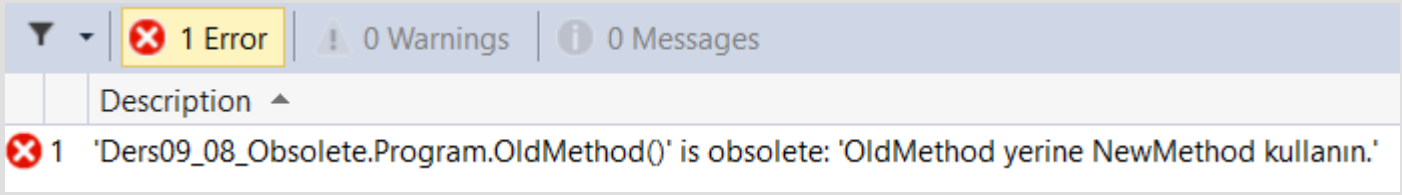
```
[Obsolete(
    message
)]
[Obsolete(
    message,
    iserror
)]
```

- *message* parametresi niye obsolote işleminin yapıldığının tutulduğu string türünden kısımdır.
- *İserror parametresi* Boolean bir değerdir. (seçimliktir) Eğer doğru ise derleyici kodu derlemez. Eğer yanlış ise sadece uyarı (warning) oluşturur.

Obsolete

```
class Program
{
    [Obsolete("OldMethod yerine NewMethod kullanın.", true)]
    static void OldMethod()
    {
        Console.WriteLine("Eski Metot");
    }
    static void NewMethod()
    {
        Console.WriteLine("Yeni Metot");
    }
    public static void Main()
    {
        OldMethod();
        Console.ReadKey();
    }
}
```

Parametre true olduğu için çalışmadı. Aşağıdaki uyarıyı verdi.



Parametre false olduğunda sadece uyarı verir. Warning message'larının derleyicinin çalışmasını etkilemediğini söylemiştik. (Örneğin değişken tanımlayıp hiç kullanmama durumda warning oluşur.)

Custom Built Attributes

- .Net Frawrok'ü bildirimli bilgi depolama ve çalışma zamanında bunları almak için kendinize özel Attribute'lerde tanımlamanıza olanak sağlar.
- Bu bilgi kod tasarımıza ve uygulama ihtiyaçlarına bağlı olarak herhangi bir hedef elemente ilişkili olabilir.
- Kişisel Attribute yaratma ve kullanma 4 adımda içerir.
  - Bir custom attribute tanımlama
  - Custom attribute'i inşa etme
  - Hedef program elementi'ne Custom attribute uygulama
  - Reflection üzerinden Attribute'lere erişim
- Son adım, çeşitli gösterimler bulmak için metadata üzerinden okumak için basit bir program yazmayı içerir. Metadata diğer verileri tanımlamak için kullanılan veri veya bilgi ile ilgili verilerdir. Bu program zamanında özelliklerini erişmek için reflections (yansımaları) kullanmalısınız.
- Reflection konusu önümüzdeki haftalarda anlatılacaktır.

Custom Built Attributes

```
//custom attribute'ın bir sınıf ve üyelerine atanacak
[AttributeUsage(AttributeTargets.Class |
AttributeTargets.Constructor |
AttributeTargets.Field |
AttributeTargets.Method |
AttributeTargets.Property,
AllowMultiple = true)]
```

Oluşturulacak Attribute Class'ın erişimleri belirleniyor.

Attribute Class'ından kalıtım yapılıyor.

İlk iki aşama

```
public class DeBugInfo : System.Attribute
{
    private int bugNo;
    private string developer;
    private string lastReview;
    public string message;
    public DeBugInfo(int bg, string dev, string d)
    {
        this.bugNo = bg;
        this.developer = dev;
        this.lastReview = d;
    }
    public int BugNo
    {
        get{ return bugNo; }
    }
    public string Developer
    {
        get{ return developer; }
    }
    public string LastReview
    {
        get{ return lastReview; }
    }
    public string Message
    {
        get { return message; }
        set { message = value;}
    }
}
```

Custom Built Attributes

```
[DeBugInfo(45, "Zara Ali", "12/8/2012", Message = "Return type mismatch")]
[DeBugInfo(49, "Nuha Ali", "10/10/2012", Message = "Unused variable")]
class Rectangle
{
    //Değişkenler
    protected double length;
    protected double width;
    public Rectangle(double l, double w)
    {
        length = l;
        width = w;
    }
    [DeBugInfo(55, "Zara Ali", "19/10/2012", Message = "Return type mismatch")]

    public double GetArea()
    {
        return length * width;
    }
    [DeBugInfo(56, "Zara Ali", "19/10/2012")]

    public void Display()
    {
        Console.WriteLine("Length: {0}", length);
        Console.WriteLine("Width: {0}", width);
        Console.WriteLine("Area: {0}", GetArea());
    }
}
```

3. Adım

Constructor 3 parametreli olmasına rağmen 4 parametreli tanımlamalar vardır. Message property'si set olduğu için erişilebilmiştir. Diğer değişkenlere constructor üzerinden erişilmiştir.

Reflection (Yansıma)

- Reflection nesneleri çalışma anında tip bilgileri elde etmek için kullanılır. Çalışan bir programın metadata'sına erişim veren sınıflar System.Reflection namespace'nde yer alır.
- System.Reflection namespace'i uygulama hakkında bilgiler elde etmeyi ve dinamik olarak tip, değer ve nesneleri uygulamaya eklemeyi sağlayan sınıfları içerir.

Reflection Uygulamaları

- Reflection'ın uygulama içinde kullanıldığı yerler:
  - Attritube bilgisini çalışma zamanında ulaşmamızı sağlar.
  - Çeşitli tipleri incelemeyi sağlar.
  - Method ve property'lere geç bağlanmayı sağlar.
  - Çalışma zamanında yeni tipler yaratmayı sağlar ve bu tipleri ile bazı görevler gerçekleştirir.

Reflection - Metadata

- System.Reflection sınıfının MemberInfo nesnesi bir sınıf ile ilgili attribute'leri keşfetmek için kullanılır.
- Bunun için, hedef sınıfın nesnesi tanımlamalısınız:

```
System.Reflection.MemberInfo info = typeof(MyClass);
```

```
[AttributeUsage(AttributeTargets.All)]
public class HelpAttribute : System.Attribute
{
    public readonly string Url;

    public string Topic //property, parameter
    {
        get  { return topic; }
        set  { topic = value; }
    }

    public HelpAttribute(string url)   //method, positional parameter
    {
        this.Url = url;
    }
    private string topic;
}

[HelpAttribute("Information on the class MyClass")]
class MyClass
{
}
```

Attribute sınıfından kalıtım

Attribute tanımlama

Reflection - Metadata

```
[AttributeUsage(AttributeTargets.All)]
public class HelpAttribute : System.Attribute
{
    public readonly string Url;

    public string Topic //property, parameter
    {
        get  { return topic; }
        set  { topic = value; }
    }

    public HelpAttribute(string url)   //method, positional parameter
    {
        this.Url = url;
    }
    private string topic;
}

[HelpAttribute("Information on the class MyClass")]
class MyClass
{
}
```

```
class Program
{
    static void Main(string[] args)
    {
        System.Reflection.MemberInfo info = typeof(MyClass);
        object[] attributes = info.GetCustomAttributes(true);
        for (int i = 0; i < attributes.Length; i++)
        {
            System.Console.WriteLine(attributes[i]);
        }

        Console.ReadKey();
    }
}
```

Reflection tanımlama

Reflection üzerinden Attribute'ler diziye aktarma

Attribute'leri gösterme

Reflection

- Custom Built Attributes bölümünde bir örneği reflection konusu olmadan verilere ulaşamayacağımızı söylemiştik. Şimdi reflection ile o örneğin attribute değerlerine ulaşalım.

```
static void Main(string[] args)
{
    Rectangle r = new Rectangle(4.5, 7.5);
    r.Display();
    Type type = typeof(Rectangle);

    //Rectangle class'ın attribute'lerinin tümü
    foreach (Object attributes in type.GetCustomAttributes(false))
    {
        DeBugInfo dbi = (DeBugInfo)attributes;
        if (null != dbi)
        {
            Console.WriteLine("Bug no: {0}", dbi.BugNo);
            Console.WriteLine("Developer: {0}", dbi.Developer);
            Console.WriteLine("Last Reviewed: {0}", dbi.LastReview);
            Console.WriteLine("Remarks: {0}", dbi.Message);
        }
    }
    //method attibute'lerin tümü
    foreach (MethodInfo m in type.GetMethods())
    {
        foreach (Attribute a in m.GetCustomAttributes(true))
        {
            DeBugInfo dbi = (DeBugInfo)a;
            if (null != dbi)
            {
                Console.WriteLine("Bug no: {0}, for Method: {1}", dbi.BugNo, m.Name);
                Console.WriteLine("Developer: {0}", dbi.Developer);
                Console.WriteLine("Last Reviewed: {0}", dbi.LastReview);
                Console.WriteLine("Remarks: {0}", dbi.Message);
            }
        }
    }
    Console.ReadKey();
}
```

Attribute içeriğine ulaşım

GetMethods ile tüm metotlara ulaşma

Attribute içeriğine ulaşım

Attribute + Reflection

- Attribute ve Reflection konusunda daha iyi anlamak için aşağıdaki web sitesini öneririm.

http://www.enterprisecoding.com/post/oznitelikler-ve-reflectionla-csharp-dilini-ihtiyaclariniz-dogrultusunda-genisletin

Kaynaklar

- http://www.tutorialspoint.com/csharp/
- http://www.dotnetperls.com/exception
- http://www.buraksenyurt.com/post/C-Temelleri-Nitelikleri(Attributes)-Kavramak-bsenyurt-com-dan.aspx